Introduction

Welcome to the LBANK API! You can use this API to get contract market data, place trades, and manage your account. This document is version v1 and will be updated continuously.

The interface is the basis for providing services, and the API is divided into three categories: account, transaction and market. After creating an account on the website, developers can create APIs with different permissions according to their own needs, and use the APIs for automatic transactions or cash withdrawals.

Account and transaction APIs require identity verification and provide functions such as placing orders, canceling orders, and querying order and account information. The market quotation API provides market quotation data, and all market quotation interfaces are public.

process

If the developer needs to use the API, please apply for the API key first, and check the information such as contract permissions and click Apply API Key, and then according to this document The details are developed, and if you have any questions or suggestions during use, please give feedback in time.

API Key includes the following two parts

- `API Key` API access key
- `Secret Key` The key used for signature authentication and encryption (only visible when applying)

LBank supports two signature methods: RSA and HmacSHA256

- The RSA scheme is a digital signature scheme based on mathematically difficult problems and an identity authentication scheme for asymmetric cryptography. The sender performs the agreed encoding and operation on the message message, the user's private key, etc. to obtain a digital signature. The receiver determines the identity and the integrity of the message message through digital signature and user registration public key. In contrast, since the receiver does not need the sender's private key information during the verification process of the RSA scheme, it can be guaranteed that there is and only the sender can send legitimate messages. Due to the use of asymmetric encryption technology, RSA has high security, but the encryption and decryption speed is slow.
- The HMAC scheme is a message authentication scheme based on a hash function and a symmetric mode encryption scheme. The sender obtains the message authentication code after performing the agreed encoding and operation on the message message and the shared key. The receiver uses the message authentication code to determine the identity of the sender and the integrity of the message. The HMAC scheme is based on the hash function and has the characteristics of high security and high efficiency. Its encryption and decryption speed is fast, but because its key needs to be shared between the client and the server, the security is not as high as RSA.

Users can choose different signature authentication methods according to their actual needs.

APIs that are not bound to an IP are only valid for 30 days

The two keys, API Key and Secret Key, are closely related to account security, and should not be disclosed to others at any time.

---

access instructions

---

access URL

REST

https://lbkperp.lbank.com/

WEBSOCKETS

wss://lbkperpws.lbank.com/ws

Interface Classification

Public interface: /cfd/openApi/v1/pub

Signature authentication

API requests are very likely to be tampered with during transmission over the internet. In order to ensure that the request has not been changed, all private interfaces except the public interface (basic information, market data) must use your API Key for signature authentication to verify Whether the parameter or parameter value was changed in transit. Each API Key requires appropriate permissions to access the corresponding interface. Every newly created API Key needs to be assigned permissions. Permission types are divided into: transaction, read-only, and withdrawal. Before using an interface, please check the permission type of each interface and confirm that your API Key has the corresponding permission.

REQUEST HEADER SETTINGS

For post requests, the contentType in the request header information needs to be uniformly set to: 'application/json'.

`contentType: 'application/json'`

Add the `timestamp` parameter to the request header, the timestamp (milliseconds) of your request, such as: 1567833674095. Including this value in your query request helps prevent third parties from intercepting your request. It is recommended to obtain it through the /cfd/openApi/v1/pub/getTime API interface.

The request header adds `signature_method` parameter, RSA/HmacSHA256, currently both are supported.

The `echostr` parameter is added to the request header, which is a string of letters and numbers with a length between 30 and 40.

SIGNATURE PROCESS (HOW TO GENERATE `SIGN` IN API REQUEST PARAMETERS)

1. Prepare the string to be signed:

The request parameters required in each API, remove `sign`, plus `signature_method`, `timestamp`, `echostr` three parameters (these three parameters need to be consistent with those in the header), constitute the need to participate The signed string. At the same time, the string to be signed is required to be sorted according to the parameter name (first compare the first letter of all parameter names, and arrange them in abcd order, if the same first letter is encountered, then look at the second letter, and so on.) The following uses prv/account interface, and taking Java code as an example, the required signature string is

`string parameters="api_key=fb4e39e5-6a06-4291-9f80-d10176a0badd&asset=USDT&echostr=echostr12345678901234567890123456789O&productGroup=SwapU&signature_method=HmacSHA556&timestamp=9165`4

2. The MD5 digest of the character string to be signed into uppercase characters:

Convert `parameters` to MD5 digest and convert all characters to uppercase

`string preparedStr = DigestUtils.md5Hex(parameters).toUpperCase()`

3. Signature:

Sign `preparedStr` using RSA or HmacSHA256 and the secret key corresponding to api_key to get the final `sign`.

RSA method(signature_method = RSA):

Use the secret key corresponding to api_key to sign `preparedStr` using the SHA256 algorithm through RSA (encoded in Base64), and assign the final signature result to the parameter `sign`. Refer to the Java code in the shell.

RSA示例:

```
public static String RSA_Sign(String preparedStr, String secretKey) {
    try {
        PrivateKey priKey = getPrivateKey(secretKey);
        Signature signature = Signature.getInstance("SHA256WithRSA");
        signature.initSign(priKey);
        signature.update(content.getBytes(CharEncoding.UTF_8));
        byte[] signed = signature.sign();
        return new String(Base64.getEncoder().encode(signed));
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

private static PrivateKey getPrivateKey(String key) {
    PKCS8EncodedKeySpec keySpec = new PKCS8EncodedKeySpec(Base64.getDecoder().decode(key));
    PrivateKey privateKey = null;
    try {
        KeyFactory keyFactory = KeyFactory.getInstance(RSA);
```

```
            privateKey = keyFactory.generatePrivate(keySpec);
        } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
            e.printStackTrace();
        }
        return privateKey;
    }
```

HmacSHA256 method (signature_method = HmacSHA256):

Use the secret key corresponding to api_key to hash `preparedStr`, and assign the final signature result to the parameter `sign`. Refer to the Java code in the shell.

HmacSHA256示例:

```
public static String HmacSHA256_Sign(String preparedStr, String secretKey) {
    String hash = "";
    try {
        Mac sha256_HMAC = Mac.getInstance("HmacSHA256");
        SecretKeySpec secret_key = new SecretKeySpec(secretKey.getBytes(), "HmacSHA256");
        sha256_HMAC.init(secret_key);
        byte[] bytes = sha256_HMAC.doFinal(message.getBytes());
        hash = byteArrayToHexString(bytes);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return hash;
}
```

SUBMIT

Submit the signed `sign` and other required parameters. Taking the `prv/account` interface as an example, the parameters that need to be submitted are:

`api_key=fb4e39e5-6a06-4291-9f80-d10176a0badd&asset=USDT&echostr=echostr12345678901234567890123456789O&productGroup=SwapU&signature_method=HmacSHA256&timestamp=1665990154559`

sign=809133cb69a17beba0be076b99b4d90de872476e36da87978ab2889970ccd06d

REQUEST FORMAT

All API requests are sent as GET or POST.

And all request headers include setting three parameters Content-Type: application/json, timestamp: 1665990154559, signature_method: RSA, echostr: echostr12345678901234567890123456789O.

All get request parameters are in the path parameters, **RSA method requires urlencode to avoid special characters in the string being escaped into spaces** after signing. post submitted as json

RETURN FORMAT

Parameter Name	Data Type	Description
result	boolean	API interface return result, true/false
error_code	string	Error code returned by the interface
msg	string	interface return message
data	object	The interface returns the data body

test account

```
String privateKey = "093F44F700FC48F17DDB67390C895CE5";
String apikey = "fb4e39e5-6a06-4291-9f80-d10176a0badd"
String signature_method = "HmacSHA256";
```

error code

Error Code	Detailed Description
-99	System exception, please try again later
0	success
2	No record found
3	Record already exists
4	Invalid action
5	invalid value
7	Invalid Session
8	The contract product does not exist
9	User does not exist
11	No market data found
12	Field error
14	Repetitive action
18	Market orders cannot be queued
20	Order Due
21	Order exceeds capacity
22	Order already exists
24	Order does not exist
25	quote does not exist
26	Invalid contract product status
27	Invalid contract product status

Error Code	Detailed Description
30	Not enough quantity to modify
31	Insufficient positions, cannot close the position
32	Position limit
33	The asset is less than zero after closing the position
34	User position limit
35	Insufficient balance
36	Insufficient funds
37	Invalid Quantity
44	Quantity is illegal
48	The price is illegal
49	Price exceeds upper limit
50	The price exceeds the lower limit
51	No transaction authority
52	Can only close position
54	User is not logged in
56	No transaction authority
58	User does not match
59	User Re-Login
60	Invalid username or password
62	User cannot activate
65	Invalid login IP address
71	The order cannot be operated
76	Order has been suspended
77	Order has been activated
78	Order date missing
79	Order type not supported
80	User has no permission
88	User does not exist

Error Code	Detailed Description
99	Cannot perform operation for other users
100	Insufficient Margin
118	Single meter combination
139	Otc type error
172	Insufficient Leverage
175	Price must be greater than zero
176	Invalid API KEY
177	API key has expired
178	API key limit exceeded
179	Key Is Null
180	Margin rate not found
181	Repeat API Key
182	No limit price
183	Exceeded maximum query count per second
184	Order limit exceeded
185	Not enough open orders
186	Session does not exist
187	The price exceeds the first price
188	The price exceeds the buy price
189	Position already exists
190	Mark price error
191	Record parsing error
192	Repeat record
193	Exceeded the maximum trading volume
194	Less than the minimum trading volume
195	Position is less than the minimum volume
196	Trade prohibited
197	Fee does not exist

Error Code	Detailed Description
198	The number of positions exceeds the limit
199	Excessive Leverage
200	Insufficient position
201	Unable to change Pos type
10001	Authentication synchronization failed
10002	Authentication parameters lost
10003	Authentication and signature verification failed
10004	Request timed out
10005	Illegal parameter
10006	Not open path
10007	Authentication failed
10008	Key does not exist
10009	No permission
10010	Invalid signature
10011	Repeat request
10012	The request is too frequent

Basic data

Get the current time

Interface address: `/cfd/openApi/v1/pub/getTime`

Request method: `GET`

Request data type: `application/x-www-form-urlencoded`

Response data type: `*/*`

Interface description:

Get the current time

Request parameters:

no yet

Response parameters:

parameter name	parameter description	type	schema
data		object	
error_code		integer(int32)	integer(int32)
msg		string	
result		string	
success		boolean	

Response Example:

```
{
    "data": {},
    "error_code": 0,
    "msg": "",
    "result": "",
    "success": true
}
```

Query contract information list

Interface address: `/cfd/openApi/v1/pub/instrument`

Request method:`GET`

Request data type: `application/x-www-form-urlencoded`

Response data type:`*/*`

Interface description:

Query contract information list

Request parameters:

Parameter name	Parameter description	Request type	Required	Data type	schema
productGroup	product group	query	true	string	

Response parameters:

parameter name	parameter description	type	schema
baseCurrency	The target base currency	string	
clearCurrency	clearing currency	string	
defaultLeverage	default leverage	number	

parameter name	parameter description	type	schema
exchangeID	Exchange code	string	
maxOrderVolume	Maximum Order Volume	string	
minOrderCost	Minimum order amount	string	
minOrderVolume	Minimum Order Volume	string	
priceCurrency	Pricing currency	string	
priceLimitLowerValue	price lower limit	number	
priceLimitUpperValue	price upper limit	number	
priceTick	Minimum price change	number	
symbol	trading pair	string	
symbolName	trading pair name	string	
volumeMultiple	volume multiplier	number	
volumeTick	minimum amount of change	number	

Response Example:


```
[
    {
        "baseCurrency": "",
        "clearCurrency": "",
        "defaultLeverage": 0,
        "exchangeID": "",
        "maxOrderVolume": "",
        "minOrderCost": "",
        "minOrderVolume": "",
        "priceCurrency": "",
        "priceLimitLowerValue": 0,
        "priceLimitUpperValue": 0,
        "priceTick": 0,
        "symbol": "",
        "symbolName": "",
        "volumeMultiple": 0,
        "volumeTick": 0
    }
]
```

Query contract market list

Interface address: `/cfd/openApi/v1/pub/marketData`

Request method: `GET`

Request data type: `application/x-www-form-urlencoded`

Response data type: `*/*`

Interface description:

Query the list of contract quotations

Request parameters:

Parameter name	Parameter description	Request type	Required	Data type	schema
productGroup	product group	query	true	string	

Response parameters:

parameter name	parameter description	type	schema
highestPrice	Highest price in 24 hours	string	
lastPrice	latest price	string	
lowestPrice	24 hours lowest price	string	
markedPrice	marked price	string	
openPrice	24 hours opening price	string	
prePositionFeeRate	funding fee	string	
symbol	Subject code	string	
turnover	turnover amount	string	
volume	24 hours volume	string	

Response Example:

```
[
    {
        "highestPrice": "",
        "lastPrice": "",
        "lowestPrice": "",
        "markedPrice": "",
        "openPrice": "",
        "prePositionFeeRate": "",
        "symbol": "",
        "turnover": "",
        "volume": ""
    }
]
```

Get handicap

Interface address: `/cfd/openApi/v1/pub/marketOrder`

Request method:`GET`

Request data type: `application/x-www-form-urlencoded`

Response data type:`*/*`

Interface Description:

Get Handicap

Request parameters:

Parameter name	Parameter description	Request type	Required	Data type	schema
depth	depth	query	true	integer(int32)	
symbol	trading pair	query	true	string	

Response parameters:

parameter name	parameter description	type	schema
asks	Buy Order	array	MarketOrderResp
orders	order quantity	integer(int32)	
price	price	number	
volume	quantity	number	
bids	Sell	array	MarketOrderResp
orders	order quantity	integer(int32)	
price	price	number	
volume	quantity	number	
symbol	trading pair	string	

Response Example:


```
{
    "asks": [
        {
            "orders": 0,
            "price": 0,
            "volume": 0
        }
    ],
    "bids": [
        {
            "orders": 0,
            "price": 0,
            "volume": 0
        }
    ],
    "symbol": ""
}
```